



KARADENİZ TEKNİK ÜNİVERSİTESİ — BİLGİSAYAR BİLİMLERİ

Web Programlama

JavaScript'ten TypeScript'e

Hafta 3

JavaScript'ten TypeScript'e

Doç. Dr. Tolga Berber

KTÜ Fen Fakültesi, Bilgisayar Bilimleri Bölümü · <https://avesis.ktu.edu.tr/tberber>

Öğrenme fazları:

1 • Ön-öğretim

2 • İçerik

3 • Katılım

4 • Değerlendirme

5 • İzleme

1 • Ön-öğretim Neden Önemli?

Hafta 2'de yazdığınız etkinlik sayfası tamamen **statikti** — koltuğa tıklayınca renk değişmiyor, form gönderilince sayfa boşuna yenileniyordu.

- Bir diziyi fonksiyona geçirip içeride değiştirirseniz, dışarıdaki dizi de değişir — ama aynısını bir sayıyla yaparsanız dışarıdaki sayı **değişmez**: bu JS'in referans/değer ayrımıdır
- Sayı yerine metin geçirdiğinizde JavaScript hiç uyarır — hata kod **çalışırken** patlar; TypeScript aynı hatayı siz **yazarken** kırmızı çizgiyle gösterir
- Bu hafta önce dili ve tip sistemini kuruyorsunuz — Hf.4'te React tam olarak bu temelin üstüne kurulacak

YG I/II'de C/C++'ın değer/referans/pointer ayrımını, kendi `Liste<T>`'inizi ve `std::thread`'i biliyorsunuz — JavaScript/TypeScript'i bu derste *ilk kez* göreceksiniz.

Bu Hafta

🎯 Öğrenme Hedefleri

Bu hafta sonunda şunları yapabileceksiniz:

- 1 JavaScript'in dil modelini C++ ile kıyaslayarak açıklamak
- 2 ES modülleri, npm ve Vite ile bir TypeScript projesi kurmak
- 3 TypeScript tipleriyle Etkinlik/Koltuk/Bilet'i tanımlamak
- 4 Olay döngüsü, Promise ve DOM API'siyle davranış eklemek
- 5 Hafta 2'nin statik sayfasını canlandırmak

BÖLÜM 1

Ünite 1 — JavaScript Dil Modeli: Değer/Referans, Fonksiyonlar, Closure

JS Dil Modeli Modüller ve Vite TS Tip Sistemi Asenkron JS/DOM Özet

2. İçerik Değer Semantiği vs Referans Semantiği

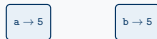
- `let/const`: blok kapsamı; `var` bu derste kullanılmaz (fonksiyon kapsamı, öngörülebilirliği düşük)
- İlkel tipler (`number`, `string`, `boolean`) bir değişkene **kopyalanarak** atanır
- Nesne ve dizi değişkenleri her zaman ortak bir belleğe işaret eden bir **referans** taşır
- C++ köprüsü: `int b = a;` bir kopyadır; JS'te nesne/dizi her zaman **T&/pointer** gibi davranır

Tanım: Değer semantiği / Referans semantiği

Değer semantiğinde bir değişken başka bir değişkene atanırken değeri kopyalanır; referans semantiğinde ise aynı bellek alanına işaret eden bir bağlantı paylaşılır.

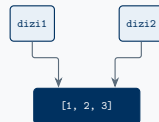
İlkel Değerler ve Nesne/Dizi Referansları

İlkel değer (kopyalanır)



let b = a; sonrası b
bağımsız bir kopyadır;
b'yi değiştirmek a'yı etkilemez.

Nesne/dizi (referans paylaşılır)



let dizi2 = dizi1; sonrası ik-
isi de AYNI diziye işaret eder,
dizi2.push(4) her ikisinde de görünür.

C++'ta `int b = a;` bir kopyadır (sol panele benzer); `std::vector<int> &v2 = v1;` bir referanstır (sağ panele benzer) — JS'te nesne/dizi değişkenleri HER ZAMAN sağdaki gibi davranır, seçim yapamazsınız.

Analoji: Fotokopi Kağıdı ve Ev Anahtarı

Analoji

Değer semantiğini bir sayıyı kağıda yazıp fotokopisini vermeye, referans semantiğini bir evin anahtarını iki kişiye vermeye benzetin: fotokopi kağıdını (değer) değiştirmek orijinali etkilemez; ama aynı evin anahtarını (referans) iki kişiye verirseniz, biri evdeki bir eşyayı taşırsa diğeri de bu değişikliği görür.

JS'te nesne/dizi değişkenleri her zaman anahtar gibi davranır — “kopya” seçeneğiniz yoktur.

Kod: Değer mi, Referans mı?

```
1 // İlkel değer: kopyalanarak atanır (C++'taki "int b = a;" gibi bir kopya).
2 let a = 5;
3 let b = a;
4 b = 10;
5 console.log("a:", a); // 5 – b'nin değişmesi a'yı etkilemez
6
7 // Nesne/dizi: değişken her zaman ortak belleğe işaret eden bir referans
8 // taşır (C++'taki T&/pointer semantiğine benzer; "kopya" seçeneği yoktur).
9 let dizi1 = [1, 2, 3];
10 let dizi2 = dizi1;
11 dizi2.push(4);
12 console.log("dizi1:", dizi1); // [1, 2, 3, 4] – dizi2 ile AYNI diziyi işaret eder
13
14 // Aynı fikir fonksiyon parametresinde de geçerlidir: dizi bir referans
15 // olarak geçirilir, fonksiyon içindeki değişiklik dışarıdan da görünür.
16 function ekle(dizi) {
17   dizi.push(99);
18 }
19 let sayilar = [1, 2, 3];
20 ekle(sayilar);
21 console.log("sayilar:", sayilar); // [1, 2, 3, 99]
```

`dizi2 = dizi1` aynı diziyi işaret eden ikinci bir referans oluşturur — `push` her iki değişkenden görünen tek diziyi değiştirir.

2 · İçerik Fonksiyon, Ok Fonksiyonu, Closure, Prototip

- Fonksiyon bildirimi (`function ad() {}`) vs fonksiyon ifadesi (`const ad = function() {}`)
- Ok fonksiyonu (`(x) => x * 2`): kısa söz dizimi, `this` bağlamını tanımlandığı yerden alır (leksik `this`)
- Closure: bir iç fonksiyon, dış fonksiyon kapandıktan sonra bile o kapsamdaki değişkenlere erişmeye devam eder
- Prototip zinciri: bir nesnede olmayan özellik prototipte aranır; `class` bu mekanizmanın okunaklı bir kısayolu
- Dinamik tiplendirme: bir değişkenin türü çalışma zamanında değişebilir — Ünite 3'ün TypeScript'i tam olarak bu yüzden gerekir

Analoji: Dolap Anahtarı ve Aile Soyadı

Analoji

Closure'ı bir dolabın anahtarını yalnızca ona ihtiyacı olan görevliye vermeye benzetin: iç fonksiyon, oluşturulduğu “oda”daki dolabın anahtarını elinde tutar; oda görünürde kapansa bile (dış fonksiyon bitse bile) o dolaba erişmeye devam eder.

Analoji

Prototip zincirini bir ailede soyadı sormaya benzetin: kişide yoksa önce ebeveynine, o da yoksa büyük ebeveynine bakılır — JS de bir nesnede bulunmayan özelliği önce prototipinde arar.

Kod: Bağımsız Sayaçlar Üreten Closure

```
1 // olusturSayac(), her çağrıldığında kendi bağımsız "deger" değişkenine
2 // sahip YENİ bir fonksiyon döndürür. Dönen iç fonksiyon, dış fonksiyon
3 // (olusturSayac) çoktan çalışmasını bitirmiş olsa bile "deger"e erişmeye
4 // devam eder – bu, closure'ın tanımıdır.
5 function olusturSayac() {
6   let deger = 0;
7   return function () {
8     deger++;
9     return deger;
10  };
11 }
12
13 const sayac1 = olusturSayac();
14 const sayac2 = olusturSayac();
15
16 console.log("sayac1():", sayac1()); // 1
17 console.log("sayac1():", sayac1()); // 2
18 console.log("sayac2():", sayac2()); // 1 – sayac1'den bağımsız kendi deger'i
19 console.log("sayac1():", sayac1()); // 3
```

`sayac1` ve `sayac2` birbirinden bağımsız kendi `deger`'ini tutar — ikisi de aynı `olusturSayac()` çağrısından üretildiği hâlde.

BÖLÜM 2

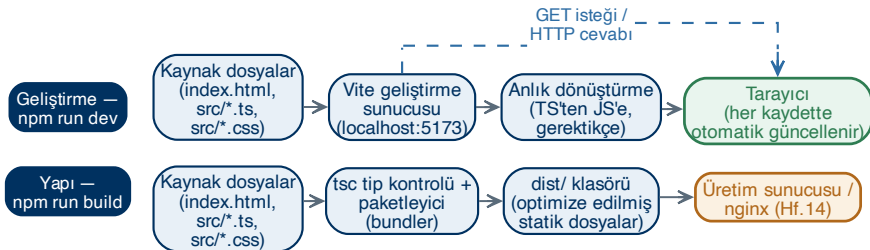
Ünite 2 — Modüller ve Araç Zinciri: ES Modülleri, npm, Vite

JS Dil Modeli **Modüller ve Vite** TS Tip Sistemi Asenkron JS/DOM Özet

2 · İçerik ES Modülleri, npm, package.json

- ES modülleri: `export/import` ile bir dosya kendi kapsamını korur — global kapsam kirliliğini önler
- Node.js: tarayıcı dışında JS/TS çalıştıran çalışma zamanı — bu haftanın araç zinciri (`npm`, `vite`) Node üzerinde çalışır
- `package.json`: bağımlılıklar (`dependencies`) ve geliştirme bağımlılıkları (`devDependencies`); `scripts` altında `dev/build/preview` kısayolları

Vite'in Geliştirme ve Yapı Akışı



Analoji: Ödünç Kitap ve Alışveriş Listesi

Analoji

ES modülünü bir kütüphaneden ödünç kitap almaya benzetin: her dosya kendi rafını (özel kapsamını) korur; yalnızca “ödünç verilebilir” (export edilen) kısımlar başka dosyalarca alınabilir (import edilebilir).

Analoji

npm ile `package.json`'u bir markete alışveriş listesiyle gitmeye benzetin: liste (`package.json`) hangi paket, hangi sürüm demektir; npm bu listeye göre kayıt defterinden paketleri toplayıp projenize getirir.

Vite ile Proje Kurulumu

>_ Terminal

```
$ npm create vite@latest -- --template vanilla-ts  
$ cd istemci && npm install  
$ npm run dev
```

- Bu komut **React şablonunu değil**, saf TypeScript şablonunu seçer — React Hf.4'te aynı klasöre eklenecek
- `npm run dev`: anlık (hot module replacement) bir geliştirme sunucusu, `http://localhost:5173`
- `npm run build`: `tsc` tip kontrolünden geçirip optimize edilmiş `dist/` klasörü üretir — geliştirme sunucusu üretimde asla kullanılmaz

BÖLÜM 3

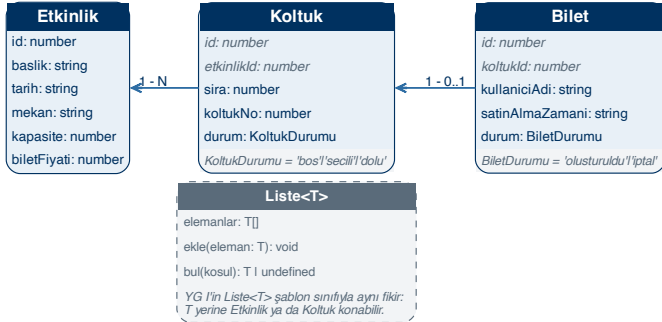
Ünite 3 — TypeScript Tip Sistemi: Etkinlik, Koltuk, Bilet

[JS Dil Modeli](#) [Modüller ve Vite](#) **[TS Tip Sistemi](#)** [Asenkron JS/DOM](#) [Özet](#)

2. İçerik Temel Tipler, Tip Çıkarımı, interface/type

- Temel tipler: `string`, `number`, `boolean`, dizi tipi `T[]`; `any` tip güvenliğini tamamen kapatır — kaçının
- Tip çıkarımı: her değişkene tip yazmak zorunlu değildir — TypeScript ilk atanan değere bakıp tipi otomatik çıkarır
- `interface/type`: bir nesnenin “şekli” (alan adları + tipleri) tanımlar
- Union ve literal tipler: `'bos' | 'secili' | 'dolu'` gibi sınırlı bir değer kümesi
- `readonly`: bir alanın ilk atamadan sonra değiştirilemeyeceğini belirtir — `id` alanları için doğal

Etkinlik / Koltuk / Bilet Alan Modeli



Analoji: Form Şablonu ve Kapı Girişi

Analoji

`interface/type`'ı bir form şablonuna benzetin: bir başvuru formu hangi alanların (ad, tarih, sayı) hangi tipte doldurulacağını önceden belirler — bir `interface` de bir nesnenin hangi alanlara ve hangi tiplere sahip olması gerektiğini önceden belirler.

Analoji

Union tipini bir kapıya iki türden biriyle girebilmeye benzetin (kart ya da şifre): bir değişken önceden tanımlı birkaç olası değerden yalnızca birine sahip olabilir, üçüncü bir seçenek kabul edilmez.

Kod: Koltuk Arayüzü — readonly ve Union

```
1 export type KoltukDurumu = 'bos' | 'secili' | 'dolu';
2
3 export interface Koltuk {
4   readonly id: number;
5   readonly etkinlikId: number;
6   sıra: number;
7   koltukNo: number;
8   durum: KoltukDurumu;
9 }
```

- `readonly id/etkinlikId`: ilk atamadan sonra değiştirilemez
- `durum: KoltukDurumu` yalnızca üç değerden birini alabilir — `'rezerve'` atarsanız `tsc` hata verir

Kod: Generic — Liste<T>

```
1 // YG I'in kendi Liste<T> şablon sınıfıyla doğrudan kıyas: bu arayüz  
  de  
2 // hangi tip için çalışacağını KULLANIM ANINDA (Liste<Etkinlik>,  
3 // Liste<Koltuk> gibi) belirleyen bir kalıptır.  
4 export interface Liste<T> {  
5   elemanlar: T[];  
6   ekle(eleman: T): void;  
7   bul(id: number): T | undefined;  
8 }
```

- YG I'in kendi `Liste<T>` şablon sınıfıyla doğrudan kıyas — hangi tip için çalışacağı kullanım anında (`Liste<Koltuk>`) belirlenir
- `npx tsc --noEmit`: kodu çevirmeden yalnızca tip hatalarını denetler

BÖLÜM 4

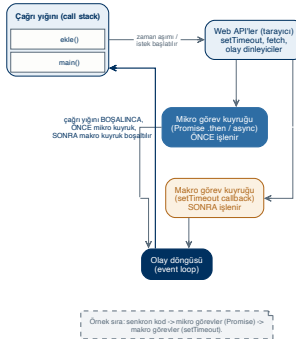
Ünite 4 — Asenkron JavaScript ve DOM: Olay Döngüsü, Promise, async/await

JS Dil Modeli Modüller ve Vite TS Tip Sistemi **Asenkron JS/DOM** Özet

2 · İçerik Olay Döngüsü: Tek İş Parçacığı

- JS **tek** iş parçacığında çalışır — çağrı yığını tamamen boşalmadan görev kuyruğundaki hiçbir iş alınmaz
- `setTimeout(fn, 0)` bile “hemen” değil “sıra gelince” demektir
- Mikro görev kuyruğu (`Promise`) makro görev kuyruğundan (`setTimeout`) önce boşaltılır
- YG II köprüsü: `std::thread` gerçek paralel iş parçacıkları açar; JS’te “eşzamanlılık” gerçek paralellik **değildir**

Çağrı Yığını, Görev Kuyrukları ve Olay Döngüsü



Analoji: Kasiyer Sırası ve Kuru Temizleme Makbuzu

Analoji

Olay döngüsünü tek kasiyerli bir markette bekleme sırasına benzetin: kasiyer (JS'in tek iş parçacığı) aynı anda yalnızca bir müşteriyle ilgilenir; gecikmeli işler bir bekleme sırasına alınır, kasiyer eldeki müşteriyi bitirmeden sıradakine geçmez.

Analoji

Promise'i bir kuru temizlemeciye bırakılan makbuzla benzetin: işiniz hemen bitmez ama elinizdeki makbuz size ileride ya "hazır" ya da "sorun çıktı" bilgisini kesin olarak verecektir.

2 • İçerik DOM API ile Hf.2 Sayfasını Canlandırmak

- `document.querySelector/querySelectorAll`: bir öğeyi seçer
- `addEventListener`: bir olayı (`click`, `submit`) dinler
- `createElement` + `append`: yeni bir öğe oluşturup sayfaya ekler
- Etkinlik listesi artık `Etkinlik[]` dizisinden DOM ile üretilir; koltuklar tıklanabilir hâle gelir

Kod: Koltuğa Tıklama — Durum Geçişi

```
1 // Bir koltuğa tıklandığında durum geçişini uygular: bos -> secili ya da
2 // secili -> bos. dolu koltuklar tıklanamaz (hiçbir şey yapılmaz).
3 function koltukTiklamaIsleyici(
4   koltuk: Koltuk,
5   eleman: HTMLDivElement,
6   degisiklikOldu: () => void
7 ): void {
8   if (koltuk.durum === 'dolu') {
9     return;
10  }
11
12  koltuk.durum = koltuk.durum === 'bos' ? 'secili' : 'bos';
13  eleman.classList.toggle('secili', koltuk.durum === 'secili');
14  degisiklikOldu();
15 }
```

`dolu` koltuklar tıklanamaz; aksi hâlde durum `bos` ↔ `secili` arasında geçiş yapar.

Kod: Form Gönderimi — preventDefault + async

```
1 // Formun submit olayında preventDefault() çağrılır (sayfa yenilenmez);
2 // seçim özeti konsola basılır, ardından async biletOlustur() çağrılıp
3 // sonucu bekleyen bir onay logu yazılır.
4 export function biletFormBaglan(
5   form: HTMLFormElement,
6   adSoyadGirdi: HTMLInputElement,
7   koltuklar: Koltuk[]
8 ): void {
9   form.addEventListener('submit', (olay) => {
10     olay.preventDefault();
11
12     const seciliKoltuklar = koltuklar.filter((k) => k.durum === 'secili');
13     const ozet: SecimOzeti = {
14       kullanıcıAdi: adSoyadGirdi.value,
15       koltukIdleri: seciliKoltuklar.map((k) => k.id),
16     };
17
18     console.log('Bilet özeti:', ozet);
19
20     biletOlustur(ozet).then((bilet) => {
21       console.log('Bilet oluşturuldu:', bilet);
22     });
23   });
24 }
```

BÖLÜM 5

Özet

[JS Dil Modeli](#) [Modüller ve Vite](#) [TS Tip Sistemi](#) [Asenkron JS/DOM](#) **[Özet](#)**

Özet

Özet

- Değer/referans semantiği + fonksiyon/closure/prototip: JavaScript'in dil modelini C++ ile kıyaslayarak kavradınız
- ES modülleri + npm + Vite: bir TypeScript projesini kurup geliştirme/derleme akışını çalıştırdınız
- `interface/type` + union + `readonly` + generic: Etkinlik/Koltuk/Bilet tiplerini tanımlayıp `tsc --noEmit` ile doğruladınız
- Olay döngüsü + Promise/`async-await` + DOM API: Hf.2'nin statik sayfasına ilk kez davranış eklediniz

5 • İzleme Aktarım — Başka Nerede?

- Bu haftanın `Etkinlik[]/Koltuk[]` dizileri birer nesne dizisidir — Hf.4'te React'in `state` güncellemesinde “mutasyon” ile “yeni dizi” farkı bu referans semantiğine dayanır
- Bugün kurduğunuz `istemci/` projesi Hf.4'te React'le büyüyecek; Hf.14'te `npm run build` çıktısı Docker+nginx ile sunulacak
- `Etkinlik/Koltuk/Bilet` arayüzleri Hf.4'te React `props` tipi, Hf.6'da FastAPI'nin Pydantic modeliyle aynen kullanılacak
- Bugün elle yazdığınız koltuk tıklama/form mantığı ilk davranış katmanı — Hf.4'te React `state/onClick` bunu otomatikleştirecek

Sırada Ne Var?

Bu hafta lab'da

Hafta 2'nin statik etkinlik sayfasına — iskelet, kartlar, koltuk planı, form — baştan sona kendi elinizle TypeScript ile davranış katacak, `npm run dev` ile tarayıcıda canlı test edeceksiniz.

- **Kendinizi Sınavın** her üniteyi kapsıyor: dizi referansı neden değişir, `npm run dev` ile `npm run build` farkı, `'reserve'` ataması neden hata verir, `setTimeout(fn, 0)` neden hemen çalışmaz
- Gelecek hafta: **React I — Bileşenler** — bu haftanın elle yazdığınız DOM güncelleme kodu, React'in `state/props` ile otomatikleştirdiği bileşenlere dönüşecek